

Report 10 – Confronto QALS - GUROBI

September 25, 2026

Contents

1	Limiti Gurobi	2
2	Limiti Teorici Embedding	2
2.1	Ma allora quali sarebbero i vantaggi di usare <code>EmbeddingComposite</code> ?	2
3	Limiti Reali Embedding	3

1 Limiti Gurobi

Per quanto riguarda i limiti dell'ottimizzatore Gurobi, nella documentazione ufficiale si può notare come, oltre alle limitazioni imposte dalle possibili licenze, l'unico reale limite è relativo alla potenza del dispositivo che si sta usando. Tramite test eseguiti con un processore AMD Ryzen 5 PRO 5650U with Radeon Graphics il limite a cui sono arrivato è un problema con 132 items in un tempo di 5779,47", dopodichè il programma va in crash.

Ne conviene che non è adeguato definire un confronto in relazione a ciò, ma piuttosto è necessario discutere in relazione al tempo impiegato rispetto a QALS. Sulla base dei seguenti risultati dati dalla media su 10 esecuzioni:

N. items	QALS (s)	Gurobi (s)
72	22.42	21.83
73	21.71	21.40
74	19.73	30.53
75	23.12	25.12
76	22.47	77.87
77	24.4	88.4

Table 1: Confronto dei tempi di esecuzione tra QALS e Gurobi al variare del numero di item.

si può concludere che non sempre vale la pena eseguire QALS: infatti fino a 73 items (incluso) conviene utilizzare Gurobi, dal 74-esimo in poi QALS sembra impiegarci meno tempo.

2 Limiti Teorici Embedding

Si consideri ora un aspetto più quantitativo, ovvero cercare di capire quali siamo, i limiti fisici dell'analealer. Per fare ciò è necessario ricordare che fino ad ora si è considerata una mappatura 1:1 sulla topologia Pegasus con 5700 qubit teorici, utilizzabili 5436 ossia il 3% (ref: QALS_implementations). Vien da se che nel momento in cui cercassi di eseguire QALS con un problema di 5437 items fallirebbe.

Se invece si utilizzasse `EmbeddingComposite` che, anziché fare un mapping 1:1, cerca di trovare un minor embedding, cioè un processo (euristico) di mappatura delle variabili di un problema di ottimizzazione (rappresentato come un grafo logico) sulla struttura e sulle connessioni fisiche (topologia) dei qubit di un computer quantistico, il problema principale sarebbe che il minor embedding potrebbe fallire nel caso in cui il problema sia troppo denso e quindi il tetto reale massimo si abbassa proprio a causa della densità, non alternando però il tetto ideale.

2.1 Ma allora quali sarebbero i vantaggi di usare EmbeddingComposite?

Utilizzando il costume il problema che si cerca di ottimizzare cambia ogni volta e quindi quello che si sta ottimizzando non è il vero problema ma una sua versione approssimata, con `EmbeddingComposite` invece no.

3 Limiti Reali Embedding

Nella pratica usando il simulated annealing:

```
1 def generate_pegasus(n):
2     G = dnx.pegasus_graph(16)
3
4     tmp = nx.to_numpy_array(G) # 5760 x 5760
5
6     rows = []
7     cols = []
8
9     for i in range(n):
10         rows.append(i)
11         cols.append(i)
12         for j in range(n):
13             if(tmp.item(i,j)):
14                 rows.append(i)
15                 cols.append(j)
16
17     return list(zip(rows, cols))
```

si definisce una matrice di 5760 qubits, tutti utilizzabili, quindi le dimensioni massime di un problema sono 5760 items (grazie al mapping 1:1). Inoltre si crea una differenza di 324 qubits che porta non poche differenze dal vero annealing, in quanto, di quei 324 qubits, non conta solo la quantità, ma anche la loro disposizione e collegamenti con altri qubits all'interno della topologia.